

AIM2 Preparation

Week 1 - Day 4: Functions & Modular Programming

5-Minute Review Notes | def, Parameters, Arguments & return

Day 4 Goal

Learn how to organize reusable logic into functions, pass data into functions with parameters, return calculated values, and break larger programs into smaller understandable pieces.

1. What Is a Function?

A function is a reusable block of code designed to perform a specific task. Instead of repeating the same logic, define it once and call it whenever needed.

```
def greet():  
    print("Hello!")  
  
greet()
```

2. Defining vs Calling a Function

Part	Meaning
def greet():	Defines the function
greet()	Calls/runs the function

3. Parameters and Arguments

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Krishna")
```

Parameter: the variable written in the function definition (name). **Argument:** the actual value supplied when calling the function ("Krishna").

4. Multiple Parameters

```
def add(a, b):  
    return a + b  
  
result = add(10, 5)
```

Functions can receive multiple inputs. Each argument is matched to a parameter.

5. return

```
def calculate_total(math, python, ai):  
    total = math + python + ai  
    return total  
  
student_total = calculate_total(85, 92, 88)
```

return sends a value back to the code that called the function. The returned value can be stored in a variable, printed, compared, or passed to another function.

6. print() vs return

print()	return
---------	--------

Displays information to the screen	Sends a value back to the caller
Useful for output/debugging	Useful for calculations and reusable logic
Does not replace returning a result	Returned value can be stored and reused

```
def add(a, b):
    return a + b

answer = add(5, 3)
print(answer)
```

7. Calculator Function Practice

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b
```

Each function has one clear responsibility. This makes code easier to read, test, reuse, and debug.

8. Functions + Conditional Logic

```
def get_result(average):
    if average >= 50:
        return "Pass"
    else:
        return "Fail"
```

Functions can contain the control-flow logic learned on Day 3.

9. Why Functions Matter

- Avoid repeating the same code.
- Give meaningful names to pieces of logic.
- Make programs easier to read and maintain.
- Allow one piece of logic to be tested independently.
- Make larger programs easier to divide into smaller tasks.
- Prepare you for modules, classes, data pipelines, and ML functions later.

10. Modular Programming Idea

Instead of writing one very large program, divide the work into small functions:

```
calculate_total()
calculate_average()
get_grade()
display_result()
```

This is the basic idea of modular programming: separate responsibilities into reusable components.

11. Function Indentation

```
def greet(name):
    message = f"Hello {name}"
    print(message)
```

```
greet("Krishna")
```

Indented statements belong to the function. Code aligned back at the left margin is outside the function. Blank lines can improve readability, but indentation determines the actual code structure.

Day 4 Practice Highlights

- Defined functions with def.
- Called functions using parentheses.
- Used parameters and arguments.
- Returned calculated values with return.
- Built add, subtract, multiply, and divide functions.
- Combined functions with conditions.
- Practiced organizing logic into small reusable pieces.

Quick Syntax Cheat Sheet

```
def function_name():  
    ...  
  
def function_name(parameter):  
    ...  
  
def add(a, b):  
    return a + b  
  
result = add(10, 5)
```

Key Takeaways

- def creates a function.
- Calling function_name() executes the function.
- Parameters are placeholders in the definition; arguments are actual values passed during a call.
- return sends a result back to the caller.
- print() displays a value; return makes a value reusable by other code.
- Small focused functions make programs easier to understand and maintain.
- Functions are a major building block for the more advanced Python and ML code ahead.

Progress

Week 1 Day 4 - Functions & Modular Programming: COMPLETE

Practice focus: greet() and calculator functions using parameters and return values.